

DATADOG FOR SREs



FROM OBSERVABILITY FUNDAMENTALS
TO PRODUCTION RELIABILITY ENGINEERING



METRICS



LOGS



TRACES



MONITORS



SLOs



INFRASTRUCTURE



YOUR
SYSTEMS



DATADOG
OBSERVABILITY
PLATFORM



INSIGHTS
& VISIBILITY



SRE
RELIABLE
SYSTEMS



- ✓ MONITOR EVERY LAYER
- ✓ DETECT ISSUES EARLY
- ✓ INVESTIGATE FASTER
- ✓ PROTECT SLOs
- ✓ IMPROVE RELIABILITY
- ✓ DELIVER BETTER EXPERIENCES



SRE MISSION

Ensure systems are reliable, observable, and continuously improving.

OBSERVE. UNDERSTAND. RELIABILIZE.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

DATADOG EXPLAINED, WHY SREs NEED OBSERVABILITY

1 WHAT DATADOG IS



Datadog is an observability and security platform that helps SREs monitor, analyze, and understand their systems in real time.

2 MONITORING vs OBSERVABILITY

MONITORING	OBSERVABILITY
 - Tells you WHEN something is wrong - Uses predefined alerts and thresholds - Focused on known issues	 - Helps you UNDERSTAND why something is wrong - Uses telemetry data to explore and discover - Focused on unknown issues

3 WHY PRODUCTION SYSTEMS BECOME DIFFICULT TO UNDERSTAND



- Many services and dependencies
- Thousands of requests per second
- Distributed across environments
- Constant changes and releases
- Failures happen in complex ways
- Logs and metrics are scattered

4 THE 4 PILLARS OF OBSERVABILITY IN DATADOG



5 TELEMETRY EXPLAINED SIMPLY



Telemetry is data about your system. Your systems continuously send telemetry (metrics, logs, traces, events) to Datadog. Datadog stores, processes, and visualizes this data so SREs can understand and improve system health.

6 WHERE DATADOG FITS IN AN SRE STACK



7 MENTAL MODEL: APPLICATION → TELEMETRY → DATADOG → SRE

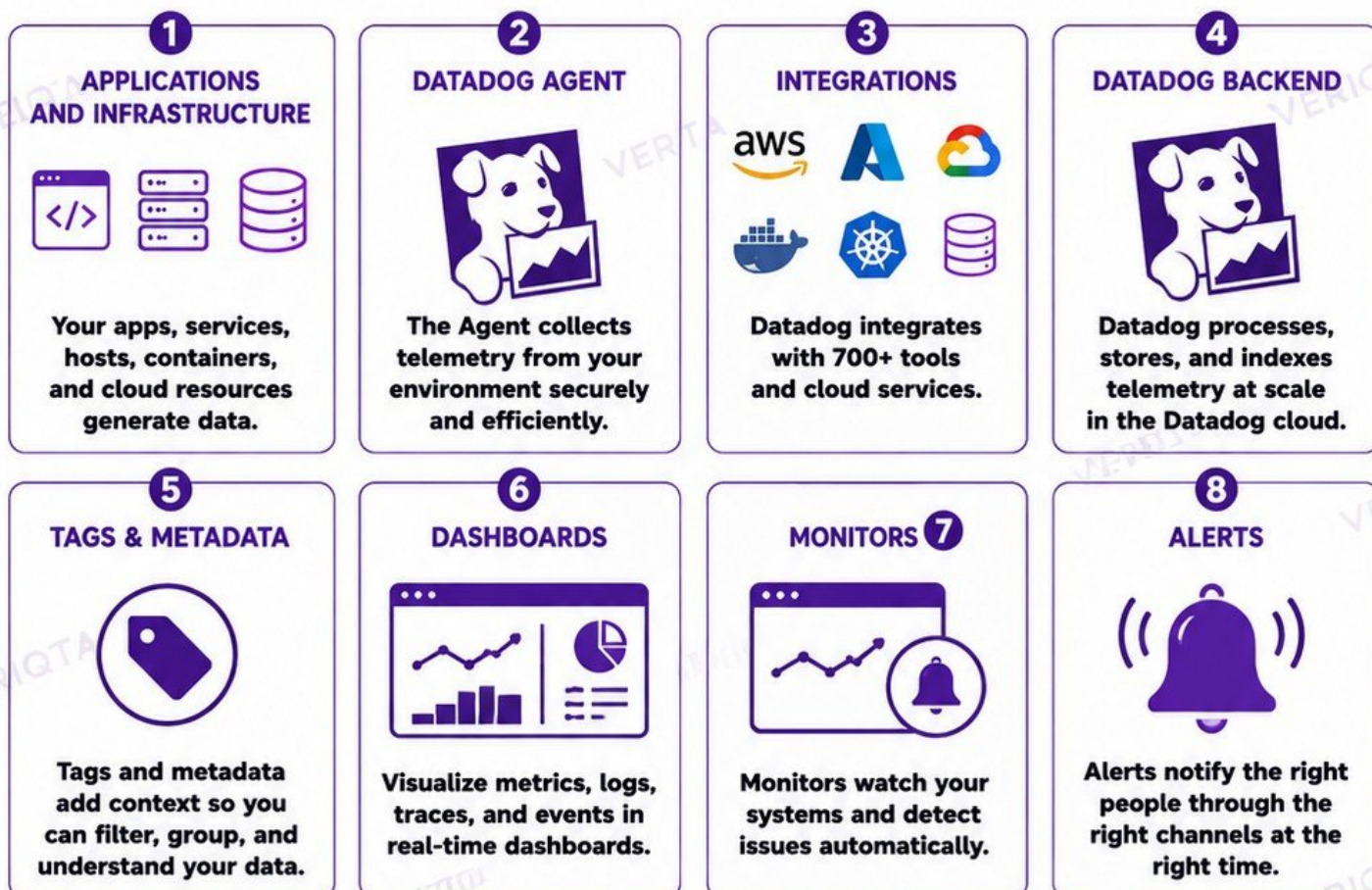


8 SRE TIP



Collecting data is not the same as understanding system health. Good observability helps you reduce unknowns and improve reliability.

HOW DATADOG WORKS, THE OBSERVABILITY PIPELINE



ARCHITECTURE



SRE TIP

Collecting data is not the same as understanding system health. Good observability turns data into actionable insights.

INSTALLING AND UNDERSTANDING THE DATADOG AGENT

1 WHAT THE AGENT DOES



The Datadog Agent runs on your infrastructure and collects metrics, logs, traces, events, and other telemetry. It then sends the data securely to Datadog.

2 DEPLOYMENT OPTIONS

HOST AGENT



Installed directly on your hosts (physical or VM).

CONTAINERIZED AGENT



Runs as a container (Docker) on your infrastructure.

KUBERNETES DEPLOYMENT



Deployed as a DaemonSet to monitor all nodes and workloads.

3 AGENT CONFIGURATION



- Main config file: `datadog.yaml`
- Enable integrations
- Set collection intervals
- Configure logs, APM, process, etc.
- Restart Agent after changes

4 API KEYS



Your Datadog API key allows the Agent to send data to your Datadog account.

Keep it secret. Treat it like a password.

5 CHECK AGENT STATUS



```
$ datadog-agent status
```

```
Agent (v7.x.x)
Status:      running ✓
Uptime:      2 days
Forwarder:   running ✓
Collector:   running ✓
```

6 VIEW AGENT LOGS



```
$ sudo tail -f /var/log/datadog/agent.log
```

Use logs to troubleshoot Agent startup, integrations, connections, and errors.

7 VERIFY TELEMETRY DELIVERY



- Check in Datadog UI → Infrastructure
- Confirm metrics are appearing
- Validate logs, traces, and events

8 COMMON AGENT FAILURES

- ! API key is invalid or missing
- ! Network/proxy blocking outbound traffic
- ! Time is out of sync on the host
- ! Insufficient permissions
- ! Disk full (logs/metrics cannot be written)
- ! Agent not running or crashed



Check agent logs first. They tell you exactly what's wrong.

9 SECURITY OF CREDENTIALS



- Store API keys securely (Vault, Secrets Manager, etc.)
- Do NOT hardcode keys in scripts or images
- Use least-privilege access
- Rotate keys if compromised
- Restrict key access to trusted users



SRE TIP

A healthy Agent means healthy observability. If the Agent is down, your visibility is down.



@veriqta



veriqta



veriqta



veriqta



@veriqta



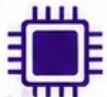
@veriqta

INFRASTRUCTURE MONITORING FOR SREs



Infrastructure monitoring gives SREs real-time visibility into the health and performance of the systems that run their services.

1 KEY HOST METRICS



CPU

Utilization and saturation over time



MEMORY

Usage, available, buffers, cache, swap



DISK

Space used, I/O, latency, inodes



NETWORK

Throughput, errors, drops, latency



LOAD

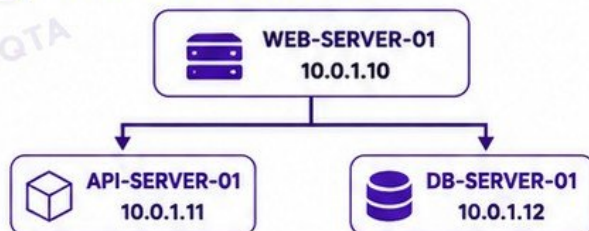
System load average and capacity



PROCESSES

Top CPU/Memory consumers and process health

2 INFRASTRUCTURE MAP



See relationships, dependencies, and health across your entire infrastructure.

3 HOST TAGS



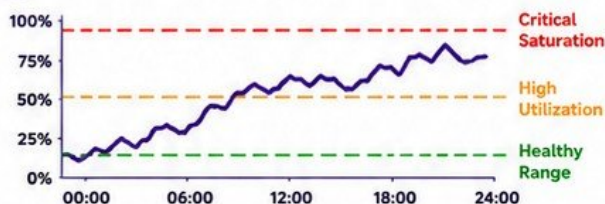
Use tags to organize and filter your hosts.

Examples:

env:prod role:web team:payments
region:us-east-1 service:checkout

Good tagging = Better filtering, smarter alerts, and faster troubleshooting.

4 DETECTING RESOURCE SATURATION



Saturation happens when demand consistently exceeds the system's capacity. Monitor trends, not just current values.

5 WHY HIGH CPU ALONE DOES NOT PROVE AN INCIDENT



- High CPU can be normal during batch jobs or peak traffic.
- Check: latency, errors, saturation, and user impact.
- Look at the full picture, not a single metric.

High CPU + High Latency + Errors = Incident
High CPU + Low Latency + No Errors = Normal



SRE TIP

Correlate metrics, logs, and traces to understand the real impact. Visibility without context is just noise.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

METRICS AND TIMESERIES, READING SYSTEM BEHAVIOR



1 WHAT A METRIC REPRESENTS

A metric is a numerical measurement of something in your system over time. It turns system behavior into data you can visualize, alert on, and act on.

2 TYPES OF METRICS

GAUGE



Shows a value at a specific moment. Can go up or down.

Example:
CPU Usage (%)

COUNT



A total that only increases.

Example:
Requests Received

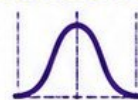
RATE



The number of events per second.

Example:
Requests per Second

DISTRIBUTION



Shows how values are spread.

Example:
Request Latency (ms)

3 AGGREGATION



Combines multiple data points into a single value for easier analysis.

Examples: avg, sum, min, max, count

4 ROLLUPS



Downsamples data over time to improve performance. Datadog rollups adjust automatically based on your time range.

5 TIME WINDOWS



The time range over which data is queried and visualized. Examples: 5m, 1h, 6h, 24h

6 PERCENTILES



Percentiles show how a value compares to others. They reveal the "tail" of your data. Used for latency, response time, etc.

7 COMMON PERCENTILES

p50

50th Percentile
Median

Half the requests are faster, half are slower.

p95

95th Percentile

Only 5% of requests are slower than this value.

p99

99th Percentile

Only 1% of requests are slower than this value.

8 WHY AVERAGES CAN HIDE PRODUCTION PROBLEMS

EXAMPLE: AVERAGE LATENCY = 200ms (Looks good?)



- Averages treat all values equally.
- A few very slow requests can ruin user experience but barely move the average.
- Always watch p95 and p99, not just the average.
- Focus on the tail, that's where users feel pain.



SRE TIP

Use the right metric type, the right aggregation, and the right percentile to see the truth in your system.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

TAGS, CARDINALITY, AND OBSERVABILITY ORGANIZATION

1 WHAT TAGS ARE



Tags are key-value pairs that add context to your telemetry. They help you filter, group, and understand your data.

format: **key:value**

2 CORE SERVICE TAGS (ALWAYS USE THESE)



env

Environment of the workload

Examples:
prod, staging, dev



service

The name of your service or application

Examples:
checkout, payment-api



version

The version or release deployed

Examples:
v1.2.3, 2024.05.01



team

Owning team of the service

Examples:
payments, platform

3 TEAM AND REGION TAGS



team

Who owns and maintains the service.



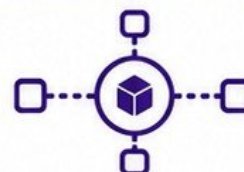
region

Where the workload is running or serving users.

OTHER USEFUL TAGS

- role: web, api, worker
- region: us-east-1, eu-west-1
- zone: us-east-1a
- cluster: prod-cluster-1
- business_unit: payments
- component: frontend, backend, db

4 UNIFIED SERVICE TAGGING



Apply the same tag keys across all services and environments. This creates consistency, better dashboards, and faster troubleshooting.

5 FILTERING AND GROUPING

FILTERING

Narrow down data to what you need.



Example:
env:prod
service:checkout

GROUPING

Break down metrics by a tag to compare multiple values.



Example:
Group by service

6 HIGH-CARDINALITY DIMENSIONS



High-cardinality tags have a very large number of unique values.

Examples:

- user_id
- session_id
- container_id
- request_id
- pod_name (auto-generated)
- timestamp

Avoid using these as tags. Use logs instead.

7 CARDINALITY: COST AND USABILITY

HIGH CARDINALITY



- More unique time series
- Higher ingestion cost
- Slower queries
- Harder to visualize
- Noisy dashboards

LOW CARDINALITY



- Fewer unique series
- Lower cost
- Faster queries
- Clear dashboards
- Better insights

8 DESIGNING A PRODUCTION TAGGING STRATEGY



- ✓ Use a consistent set of tag keys across all services.
- ✓ Keep cardinality low and intentional.
- ✓ Use tags that represent business and operations.
- ✓ Avoid tags with IDs or fast-changing values.
- ✓ Review tag usage regularly.
- ✓ Document and enforce tagging standards.



SRE TIP

Good tagging turns raw data into actionable insights.
Consistent tags = Better observability, lower cost, faster troubleshooting.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

BUILDING SRE DASHBOARDS THAT ACTUALLY HELP

Make data easy to understand. Focus on what matters.

1 DASHBOARD PURPOSE



A good dashboard answers important questions quickly and helps SREs take action.

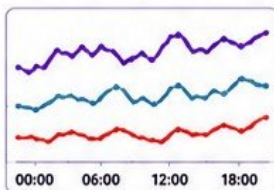
ASK YOURSELF:

- What am I trying to understand?
- Who is the audience?
- What action should this drive?
- What does "healthy" look like?

2 KEY WIDGETS IN SRE DASHBOARDS

TIMESERIES

Show trends over time.



QUERY VALUES

Show a single value or computed result.

Error Rate (5m)

2.35% ↓

vs last 5m -0.45%

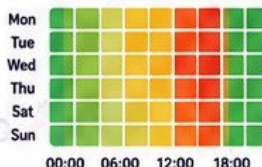
TOP LISTS

Show top items by count, latency, errors.

Top Services by Error %	
checkout	4.21%
payment	2.31%
shipping	1.12%
catalog	0.45%
user-service	0.21%

HEATMAPS

Visualize patterns across time.



TABLES

Show detailed structured data.

Service	P95 Latency	Errors
checkout	320 ms	124
payment	280 ms	98
shipping	210 ms	45
catalog	120 ms	12

SERVICE HEALTH VIEWS

Get a high-level health status across services.

checkout	↓	
payment	!	
shipping	✓	
catalog	✓	
user-service	✓	

3 THE GOLDEN SIGNALS

The foundation of most SRE dashboards.



LATENCY

How long requests take



TRAFFIC

How much is being done



ERRORS

Rate of failures



SATURATION

How full your resources are

4 AVOIDING DASHBOARD OVERLOAD



LESS IS MORE

Show only what matters. Remove noise.



AUDIENCE FIRST

Different teams need different views.



FOCUS ON ACTION

Every widget should help answer a question or drive an action.

5 EXECUTIVE DASHBOARD vs SRE OPERATIONAL DASHBOARD

EXECUTIVE DASHBOARD

High-level overview for leadership.



Business Health



SLO Status



Availability Trend



Incidents Summary



Top Impacted Services



SRE OPERATIONAL DASHBOARD

Deep-dive for engineers and responders.



Golden Signals



Infrastructure Health



Service Map



Recent Deploys



Active Alerts

6 BUILDING A PRODUCTION OVERVIEW



Define Goals and Questions



Choose Key Metrics



Pick the Right Widgets



Apply Filters and Tags



Review, Iterate and Share

7 SRE TIP



A dashboard is not a wall of charts. It is a decision-making tool. Make it simple, clear, and useful.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

MONITORS AND PRODUCTION ALERTING

Detect issues early. Alert the right people. Reduce noise.

1 WHAT IS A DATADOG MONITOR?



A monitor watches your metrics, logs, traces, or apm data and alerts you when something happens.

MONITOR = DETECTION + NOTIFICATION

2 TYPES OF MONITORS



METRIC MONITORS

Watch metrics and numeric thresholds.



LOG MONITORS

Alert on patterns in your logs.



APM MONITORS

Alert on latency, errors, or throughput.



COMPOSITE MONITORS

Combine multiple monitors with AND / OR logic.

3 THRESHOLDS



- Set the value that defines a problem.
- Static thresholds: fixed values.
- Dynamic thresholds: adjust to normal behavior.
- Example: CPU > 85% for 5m.

4 EVALUATION WINDOWS



- The time period a monitor evaluates.
- Example: last 5 minutes.
- Helps avoid false alerts from short spikes.
- Choose a window that matches the problem's impact.

5 WARNING VS ALERT



WARNING

- Something may be wrong.



ALERT

- Something is definitely wrong.

Use warnings to inform. Use alerts to page.

6 RECOVERY NOTIFICATIONS



- Get notified when the issue is resolved.
- Confirms recovery and reduces uncertainty.
- Important for closed-loop visibility.

7 ALERT ROUTING



- Send alerts to the right team or person.
- Use notification channels: Email, Slack, PagerDuty, Opsgenie, Webhooks.
- Route based on tags, severity, or team.

8 MONITOR STATES



OK
All good.



WARNING
Potential issue.



ALERT
Active problem.



NO DATA
No data received.



SILENCED
Temporarily muted.

9 AVOIDING NOISY ALERTS



- Alert on causes, not symptoms.
- Use proper thresholds and windows.
- Use anomaly detection for dynamic systems.
- Group related alerts (composite monitors).
- Silence known issues during maintenance.
- Review and refine regularly.



GOAL:

Actionable alerts that lead to fast and correct actions.

10 MONITOR FLOW



DETECT

Monitor evaluates your data.



ALERT

Notify the right people.



RESOLVE

Fix the issue.



RECOVER

Confirm and close the loop.



SRE TIP

Good alerting is not just about being notified. It's about being notified of the right things, at the right time, to the right people.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

ALERT ENGINEERING AND REDUCING ALERT FATIGUE

Better alerts. Less noise. Faster response. Happier teams.

1 ACTIONABLE ALERTS



- Alert should point to a problem that someone can fix.
- Include context, impact, and recommended next steps.

If you can't act on it, don't alert on it.

2 SYMPTOMS VS CAUSES



- Symptoms tell you something is wrong.
- Causes tell you why it's wrong.
- Alert on causes, not just symptoms.

Alert on root causes when possible.

3 STATIC THRESHOLDS



- Fixed values are simple and clear.
- Good for well-understood behaviors.
- Can create noise during normal changes.

Use when the baseline is stable.

4 DYNAMIC THRESHOLDS



- Adjusts based on historical patterns.
- Reduces noise from normal variations.
- Better for traffic that changes throughout the day.

Use when behavior changes predictably.

5 ANOMALY DETECTION



- Learns normal behavior and alerts on deviations.
- Great for unknown or complex patterns.

Detects things you didn't set thresholds for.

6 OUTLIER DETECTION



- Finds rare spikes or drops.
- Useful for unusual but important events.

Good for catching strange behavior.

7 NO-DATA BEHAVIOR



- Define what happens when data stops.
- Alert, warn, or ignore based on what makes sense.

No data can be a problem too.

8 FLAPPING ALERTS



- Alerts that fire and resolve repeatedly.
- Causes noise and erodes trust.
- Usually caused by bad thresholds or short windows.

Stabilize your alerts.

9 DEDUPLICATION



- Group related alerts into one.
- Prevents alert storms from multi-layer failures.

One incident, one alert.

10 NOTIFICATION FATIGUE



- Too many alerts = ignored alerts.
- Focus on quality, not quantity.
- Protect attention like a budget.

Noise kills signal.

11 DESIGN ALERTS AROUND USER IMPACT



- ✓ Start with the user, not the system.
- ✓ What's the impact if this fails?
- ✓ How many users are affected?
- ✓ What is our SLO or error budget impact?
- ✓ Page only when the user is in trouble.

USER IMPACT FIRST. TECHNOLOGY SECOND.

12 SRE TIP



EVERY PAGE SHOULD REQUIRE A POSSIBLE ACTION.

- Alerts exist to drive action.
- If no action is possible, rethink the alert.



REMEMBER

GOOD ALERTS ARE PRECISE, ACTIONABLE, AND USER-FOCUSED. MAKE ALERTING A FORCE MULTIPLIER, NOT A DISTRACTION.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LOG MANAGEMENT FOR PRODUCTION TROUBLESHOOTING

Collect the right logs. Find the right answers. Fix the right problems.

1 LOG COLLECTION



- Datadog Agent collects logs from your infrastructure, applications, and services.
- Logs are shipped securely to Datadog in real time.

2 SOURCES

- CLOUD SERVICES
- CONTAINERS
- HOSTS / VMS
- APPLICATIONS
- NETWORK DEVICES
- DATABASES
- THIRD-PARTY APPS

More sources = more visibility.

3 SERVICES



- Group related logs into Services.
- Helps isolate and search logs by application.
- Improves correlation across your stack.

Every Service should have clear ownership.

4 PARSING



- Parse logs to extract meaningful fields.
- Use grok patterns or built-in parsers.
- Structured logs are easier to search and analyze.

From raw text to structured data.

5 ATTRIBUTES



- Extracted fields become attributes.
- Store key information like user_id, status_code, order_id, etc.

Attributes turn logs into useful data.

6 FACETS



- Facets are attributes you can filter and group by.
- Examples: env, service, status, host, user_id.

Facets help you slice and explore fast.

7 TAGS



- Add tags to logs for context and routing.
- Examples: env:prod, service:api, team:sre.

Good tags = fast filters and better alerts.

8 SEARCHING LOGS



- Use powerful search syntax to find anything.
- Keywords, phrases, attributes, wildcards, and Boolean operators.

Right search. Right logs. Right answer.

9 FILTERING BY ENV & SERVICE



- Filter logs by environment (dev, staging, prod).
- Filter by service to focus on one application.
- Reduce noise and find what matters.

env:prod AND service:checkout show me only what I need.

10 CORRELATING LOGS WITH FAILURES



- Correlate logs with metrics, traces, and events.
- Use logs to answer WHY a failure happened.
- Follow the timeline: Alert → Logs → Root Cause

Logs explain the story behind the metrics and alerts.

11 SENSITIVE-DATA CONSIDERATIONS



- Never log secrets, PII, tokens, or passwords.
- Mask or redact sensitive data.
- Follow security and compliance policies.

Log smart. Protect users. Protect your company.

12 RETENTION & INDEXING AWARENESS



- Retention defines how long logs are stored.
- Indexing defines how fast logs can be searched.
- Higher retention and indexing = higher cost.
- Keep what you need. Archive the rest.

Right retention. Right indexing. Right cost.

SRE TIP



LOGS ARE EVIDENCE.
USE THEM TO UNDERSTAND,
NOT JUST TO SEARCH.

APM AND DISTRIBUTED TRACING FOR SRES

SEE INSIDE YOUR APPLICATION. FIND WHAT'S SLOW. FIX WHAT MATTERS.

1 APPLICATION PERFORMANCE MONITORING (APM)



APM gives you deep visibility into how your application behaves in production.

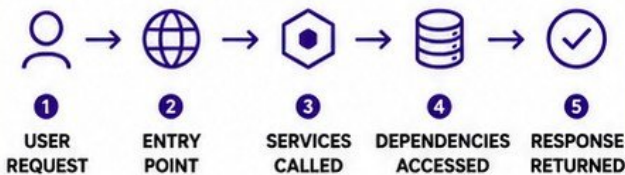
- Real user requests
- Code-level performance
- Dependencies
- Errors and exceptions

APM answers: **WHY** is my application slow or failing?

2 THE BUILDING BLOCKS

	TRACE	A complete journey of a request as it travels through your system.
	SPAN	A single operation or unit of work within a trace.
	SERVICE	A collection of resources performing similar functions (e.g., checkout-service).
	RESOURCE	The specific endpoint, method, SQL query, or external call (e.g., POST /pay).

3 REQUEST LIFECYCLE

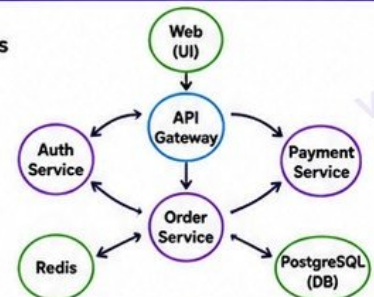


The trace shows every step and how long each took.

4 SERVICE MAP

Visualize how services interact and depend on each other.

- See traffic flow
- Spot failing services
- Identify critical dependencies



5 KEY PERFORMANCE METRICS

LATENCY  How long requests take.  Track p50, p95, p99 latency.	ERROR RATES  How often requests fail.  Monitor error % over time.	THROUGHPUT  How many requests per second/minute.  Understand load and capacity.	APDEX SCORE  User satisfaction based on response time. ● Satisfied ● Tolerating ● Frustrated Measure user happiness.
---	--	---	---

6 FINDING SLOW DEPENDENCIES

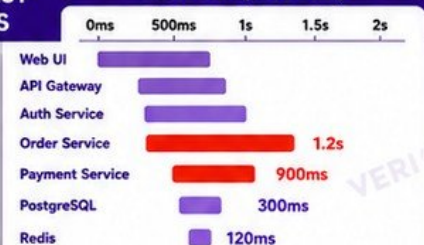
- Use Service Map to spot problem services.
- Check latency breakdown in traces.
- Identify slow external calls (DB, API, queues).
- Focus on the spans with the highest duration.

Fix the slowest dependency. It fixes the user experience.

7 FOLLOWING ONE REQUEST ACROSS MICROSERVICES

1. Start with a trace (from log, alert, or user report).
2. Follow the trace across services.
3. Drill down into spans.
4. Inspect logs and errors on slow or failed spans.
5. Find the root cause.

TRACE VIEW (WATERFALL)



SRE TIP

TRACES SHOW THE STORY. METRICS SHOW THE PATTERN. TOGETHER, THEY HELP YOU SOLVE THE PROBLEM FASTER.

CORRELATING METRICS, LOGS, AND TRACES

WHY IT MATTERS



Isolated telemetry leads to guesswork, longer MTTR, and incomplete root cause. Correlating data turns signals into answers.

CORRELATION EXAMPLE



METRIC

High error rate on checkout service



TRACE

Slow / error in payment-service



LOGS

Payment timeout from external API



INFRASTRUCTURE

High latency on DB connections



ROOT CAUSE

External payment API slowness causing checkout failures

THE INVESTIGATION FLOW

1



START FROM AN ALERT

Identify the problem and capture the context.

2



INSPECT THE METRIC

Check the metric that triggered the alert. Understand the pattern.

3



IDENTIFY THE AFFECTED SERVICE

Determine which service or component is impacted.

4



OPEN A TRACE

Look at real requests flowing through the service.

5



FIND THE FAILING SPAN

Locate slow or error spans that caused the issue.

6



CORRELATE APPLICATION LOGS

Search logs with trace ID, request ID, or error messages.

7



CHECK INFRASTRUCTURE

Review host, container, DB, and network metrics for supporting clues.

8



BUILD AN EVIDENCE-BASED HYPOTHESIS

Combine all signals to understand what happened and why.

9



VALIDATE & ACT

Confirm the root cause and take the right action.

MENTAL MODEL



ALERT



METRIC



SERVICE



TRACE



LOGS



ROOT CAUSE



SRE TIP

Don't just collect data.
Correlate it. Context is everything.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

KUBERNETES AND CONTAINER MONITORING

1 DATADOG IN KUBERNETES



Datadog provides deep visibility into your Kubernetes clusters, workloads, and applications.

2 AGENT AND CLUSTER AGENT CONCEPTS

DATADOG AGENT



Runs on each node. Collects metrics, logs, and events from containers and the host.

CLUSTER AGENT



Runs in the cluster. Collects cluster-level metrics and events from the Kubernetes API.

3 WHAT WE MONITOR IN KUBERNETES



NODES

CPU, memory, disk, network, load and more.



PODS

Health, status, restarts, CPU, memory usage.



DEPLOYMENTS

Replicas, rollout status, availability, revision history.



CONTAINERS

Per-container metrics and resource usage.



NAMESPACES

Isolate and monitor by namespace.



K8S EVENTS

API server events, scheduling issues, failures, warnings.

4 RESOURCES: REQUESTS AND LIMITS



REQUESTS

The minimum resources a container guarantees to use.



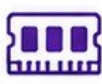
CPU

500m
2 CPU



LIMITS

The maximum resources a container is allowed to use.



MEMORY

256Mi
512Mi

5 COMMON POD FAILURE STATES



RESTARTS

Container restarts are tracked. High restart count usually means the app or config is unstable.



OOMKILLED

Container used more memory than its limit and was killed by the system.



CRASHLOOPBACKOFF

Container keeps crashing on startup. Kubernetes backs off and retries.

6 KUBERNETES TROUBLESHOOTING WORKFLOW



1 DETECT

Alert triggers or issue reported.



2 IDENTIFY

Find affected namespace, pod, or deployment.



3 INSPECT

Check metrics, pod status, and events.



4 CORRELATE

Review logs and traces for context.



5 DIAGNOSE

Determine root cause of the issue.



6 RESOLVE

Fix the issue or rollback the change.



7 VERIFY

Confirm pods are healthy.



SRE TIP

Kubernetes health = Node health + Workload health + Application health.
Monitor all layers to understand the full picture.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

CLOUD MONITORING WITH DATADOG

1 CLOUD INTEGRATIONS

Datadog integrates natively with the major cloud providers.



AMAZON WEB SERVICES
Comprehensive monitoring of AWS resources



MICROSOFT AZURE
Full visibility into Azure infrastructure and services



GOOGLE CLOUD PLATFORM
Monitor GCP resources and managed services

2 WHAT YOU CAN MONITOR IN THE CLOUD



CLOUD METRICS
Real-time metrics from cloud services and resources.



RESOURCE METADATA
Automatically collect rich resource details.



SERVICE INTEGRATIONS
700+ integrations for cloud and managed services.



CLOUD TAGS
Use cloud tags to filter, group and understand your environment.

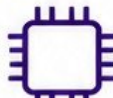
3 KEY CLOUD RESOURCES MONITORED



LOAD BALANCERS
Monitor traffic, latency, errors, and health.



DATABASES
Connections, performance, replication, and storage.



COMPUTE
CPU, memory, disk, network, and instance health.



QUEUES
Depth, message rates, failures, and processing time.



SERVERLESS AWARENESS
Monitor functions, invocations, errors, and duration.

4 CORRELATE CLOUD INFRASTRUCTURE WITH APPLICATION HEALTH



CLOUD INFRASTRUCTURE



DATADOG COLLECTS DATA



CORRELATE METRICS & LOGS



APPLICATION HEALTH



SRE INSIGHTS & ACTIONS



SRE TIP

Correlating cloud infrastructure with application telemetry helps you find the real root cause faster and reduce mean time to recovery (MTTR).



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

GOLDEN SIGNALS AND RED/USE METHODS

1 THE GOLDEN SIGNALS

The four key signals that reveal the health of your systems and user experience.



LATENCY

How long it takes to serve a request. High latency degrades user experience.



TRAFFIC

The volume of requests or work your system handles over time.



ERRORS

The rate of failed requests or operations.



SATURATION

How full your system resources are. High saturation leads to performance issues.

2 RED METHOD (APPLICATION / SERVICE LEVEL)



RATE

The number of requests per second.



ERRORS

The number or percentage of failed requests.



DURATION

The time spent serving requests (latency).

Focus: Understanding how your application behaves from the user's perspective.

3 USE METHOD (INFRASTRUCTURE LEVEL)



UTILIZATION

How busy your resources are (CPU, memory, disk).



SATURATION

How full the resources are approaching capacity.



ERRORS

The number of resource errors or failures.

Focus: Understanding how your infrastructure resources are performing.

4 CHOOSING THE RIGHT FRAMEWORK



Use RED when you care about user experience and the behavior of your application or service.

BEST FOR:

Application teams, microservices, public APIs, user-facing services.



Use USE when you care about the performance of your infrastructure resources.

BEST FOR:

Platform teams, SREs, infrastructure, clusters, databases, queues.

5 SERVICE-LEVEL vs INFRASTRUCTURE-LEVEL SIGNALS



SERVICE-LEVEL SIGNALS

- Measure what users experience
- Map to SLOs and error budgets
- Drive user-centric decisions
- Examples: latency, error rate, request rate



INFRASTRUCTURE-LEVEL SIGNALS

- Measure resource capacity and health
- Predict performance problems
- Prevent outages
- Examples: CPU usage, memory, disk I/O

6 TURNING SIGNALS INTO USEFUL DASHBOARDS AND MONITORS



1. SELECT SIGNALS
Pick the right signals for your service or infrastructure.



2. VISUALIZE
Build clear dashboards that show trends, patterns, and context.



3. ALERT SMARTLY
Create actionable monitors on symptoms that matter.



4. CORRELATE
Link metrics, logs, traces, and events for faster insights.



5. IMPROVE CONTINUOUSLY
Review, refine, and remove noise to keep observability high-value.



SRE TIP

Great observability is not about more data. It's about the right signals, in the right context, driving the right actions.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

SLI, SLO, SLA, AND ERROR BUDGETS IN DATADOG

1 KEY DEFINITIONS



SLI

SERVICE LEVEL INDICATOR

A measurable signal that indicates the health of your service from the user's perspective.

Examples: availability, latency, error rate.



SLO

SERVICE LEVEL OBJECTIVE

A target value for your SLI based on what your users need and what your business can sustain.

Example: 99.9% availability over a 30-day period.



SLA

SERVICE LEVEL AGREEMENT

A commitment to your customers, often derived from your SLOs.

Example: 99.9% uptime with credits if missed.

2 COMMON SLIs



AVAILABILITY SLI

$$\text{Availability SLI} = \frac{\text{Good Events}}{\text{Total Events}} \times 100$$

Good Event: Request succeeds.

Total Event: All requests received.



LATENCY SLI

Percentage of requests served within an acceptable latency.

$$\text{Latency SLI} = \frac{\text{Requests} \leq \text{Target}}{\text{Total Requests}} \times 100$$

3 TIME-BASED SLOS



SLOs are evaluated over a defined time window, not just minutes.

Common windows:

- 7 days
- 30 days
- 90 days

Example SLO

99.9% availability over 30 days

Allowed downtime: 43m 49s / 30 days

Objective is based on user impact and business needs, not perfection.

4 ERROR BUDGETS



Your error budget is the amount of unreliability you can afford.

$$\text{Error Budget (\%)} = 100\% - \text{SLO (\%)}$$

Example:

SLO = 99.9% → Error Budget = 0.1% (43m 49s of downtime per 30 days)

5 BURN RATE



Burn rate shows how fast you are consuming your error budget.

- Burn Rate = 1x → On track
- Burn Rate > 1x → Using budget fast
- Burn Rate >> 1x → Risk of breach

Monitor burn rate to act before the error budget is gone.

6 WHY 100% RELIABILITY IS WRONG GOAL



- Impossible and extremely costly
- Slows down releases
- Increases complexity
- Does not improve user value
- Focus on reliability that enables speed and innovation

7 CONNECTING RELIABILITY TO ENGINEERING DECISIONS



SLOs and error budgets guide important trade-offs:

- When to release
- When to roll back
- When to invest in reliability
- When to accept risk

8 SUMMARY



- Define meaningful SLIs
- Set realistic SLOs
- Track error budgets
- Monitor burn rate
- Make data-driven engineering decisions



SRE TIP

Reliability is about managing risk, not eliminating it.

Good SLOs + healthy error budgets = fast, reliable, and resilient systems.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

SLO MONITORING AND BURN-RATE ALERTING

1 SLO DASHBOARDS



- Real-time SLO status
- Error budget remaining
- Burn rate over time
- Service performance trends

2 ERROR-BUDGET CONSUMPTION



75% REMAINING
You can spend 25% more of errors this period.

3 FAST BURN vs SLOW BURN

FAST BURN

Burning error budget too quickly!



High risk of breaching SLO soon.

SLOW BURN

Burning error budget gradually.



Under control, but needs attention.

4 MULTI-WINDOW ALERTING CONCEPTS

Use multiple time windows to detect both fast and slow burns.



5 DETECT THREATS BEFORE THE BUDGET IS EXHAUSTED



- Alert early, not after the breach
- Protect user experience
- Give teams time to respond
- Prevent SLO violations

6 USER-IMPACT-FOCUSED PAGING



- Page only on user-impacting issues
- Focus on SLO and error budget
- Reduce noise and fatigue
- Actionable alerts, not noisy ones

7 RELEASE DECISIONS BASED ON ERROR-BUDGET HEALTH



- Healthy budget? Safe to release.
- High burn? Delay or fix first.
- SLO health guides engineering decisions.
- Reliability over speed.

8 PRODUCTION SCENARIO: DEPLOYMENT CAUSES LATENCY SLO DEGRADATION



DEPLOYMENT
New version deployed.



LATENCY INCREASES
p95 latency spikes above target.



BURN RATE SPIKES
Error budget burning fast!



ALERT FIRES
Fast-burn alert triggers user-impact paging.



INVESTIGATE & MITIGATE
Identify issue, mitigate or rollback.



RECOVER & VALIDATE
Latency returns to normal. Budget stabilizes.



SRE TIP

Track your error budget like cash in a bank.
Spend it wisely. Protect user trust. Deliver reliability.

INCIDENT DETECTION AND INVESTIGATION WITH DATADOG

1	ALERT FIRES		A monitor detects an issue and triggers an alert. Capture the alert details.		DATADOG MONITORS
2	ESTABLISH IMPACT		Determine user impact, severity, and business risk. Who is affected and how?		USERS & BUSINESS
3	DETERMINE AFFECTED SERVICES		Identify the services, components, and regions experiencing issues.		SERVICE MAP
4	CHECK RECENT CHANGES		Look for deployments, config changes, feature flags, or infrastructure updates.		DEPLOYMENTS & CHANGES
5	INSPECT DASHBOARDS		Open relevant dashboards for overview and real-time context.		DASHBOARDS
6	ANALYZE METRICS		Analyze key metrics, trends, latency, errors, saturation, and throughput.		METRICS
7	SEARCH LOGS		Search logs for errors, exceptions, and unusual patterns.		LOGS
8	INSPECT TRACES		Follow requests end-to-end. Find slow or failing spans.		TRACES
9	CHECK DEPENDENCIES		Verify downstream services, databases, queues, and external APIs.		DEPENDENCIES
10	FORM & TEST HYPOTHESES		Form a root cause hypothesis and test it using telemetry evidence.		HYPOTHESES
11	MITIGATE		Apply a fix, rollback, scale, or configuration change to reduce impact.		ACTIONS
12	VERIFY RECOVERY		Confirm systems are stable. Monitor key metrics and user experience.		VERIFY
13	CAPTURE INCIDENT EVIDENCE		Document timeline, impact, root cause, actions, and lessons learned.		INCIDENT REPORT

TROUBLESHOOTING MODEL



SRE TIP

Great incident response is calm, data-driven, and collaborative. Use Datadog to turn signals into clear answers.

DATADOG AS CODE AND PRODUCTION GOVERNANCE

1 WHY MANUAL CONFIGURATION CREATES DRIFT



- Manual changes are hard to track.
- Inconsistent monitors and dashboards.
- Hard to reproduce across environments.
- Risk of configuration loss.

2 MANAGING MONITORS AS CODE



- Define monitors in code (JSON/YAML).
- Consistent and repeatable.
- Easily review, version, and audit.
- Promote changes across environments.

3 MANAGING DASHBOARDS AS CODE



- Store dashboards as code.
- Version dashboards just like application code.
- Ensure consistency and traceability.

4 TERRAFORM WITH DATADOG



- Use Terraform provider for Datadog.
- Provision monitors, dashboards, SLOs, and more.
- Integrate with your IaC workflow.
- Enforce standards automatically.

5 VERSION CONTROL



- Store all observability configs in Git.
- Track every change.
- Roll back when needed.

6 PULL REQUESTS



- Propose changes via Pull Requests.
- Ensure visibility before changes are merged.

7 PEER REVIEW



- Review for correctness, best practices, and impact.
- Catch issues early.
- Share knowledge across the team.

8 ENVIRONMENT CONSISTENCY



- Same configs across all environments.
- Predictable behavior.
- Fewer surprises in production.

9 SECRETS HANDLING



- Use secure storage for API and app keys.
- Integrate with Vault, AWS Secrets Manager, or CI/CD secrets.

10 MONITOR OWNERSHIP



- Every monitor has an owner.
- Clear accountability.
- Faster response and better hygiene.

11 NAMING STANDARDS



- Use consistent prefixes, environments, and services.
- Example: [env]:[service]:[signal]:[condition]
- Improves search and clarity.

12 CHANGE HISTORY



- Full audit trail of who changed what and when.
- Easy to troubleshoot and rollback.

13 OBSERVABILITY AS PART OF PLATFORM ENGINEERING



- Observability is not optional.
- Build it into the platform.
- Provide golden signals, dashboards, and alerts as self-service.

OBSERVABILITY AS CODE WORKFLOW



1. DEVELOP
Write monitor / dashboard code



2. COMMIT
Push code to Git repository



3. OPEN PR
Create Pull Request for review



4. REVIEW
Peer review and approve



5. MERGE
Merge to main branch



6. APPLY
Terraform applies changes to Datadog



7. LIVE
Changes live in Datadog



SRE TIP

Treat your observability setup like application code. Standardize it, automate it, and govern it.

FINAL SRE PRODUCTION INCIDENT LAB

PRACTICE THE FULL SRE INCIDENT LIFECYCLE WITH DATADOG

INCIDENT LAB STEPS

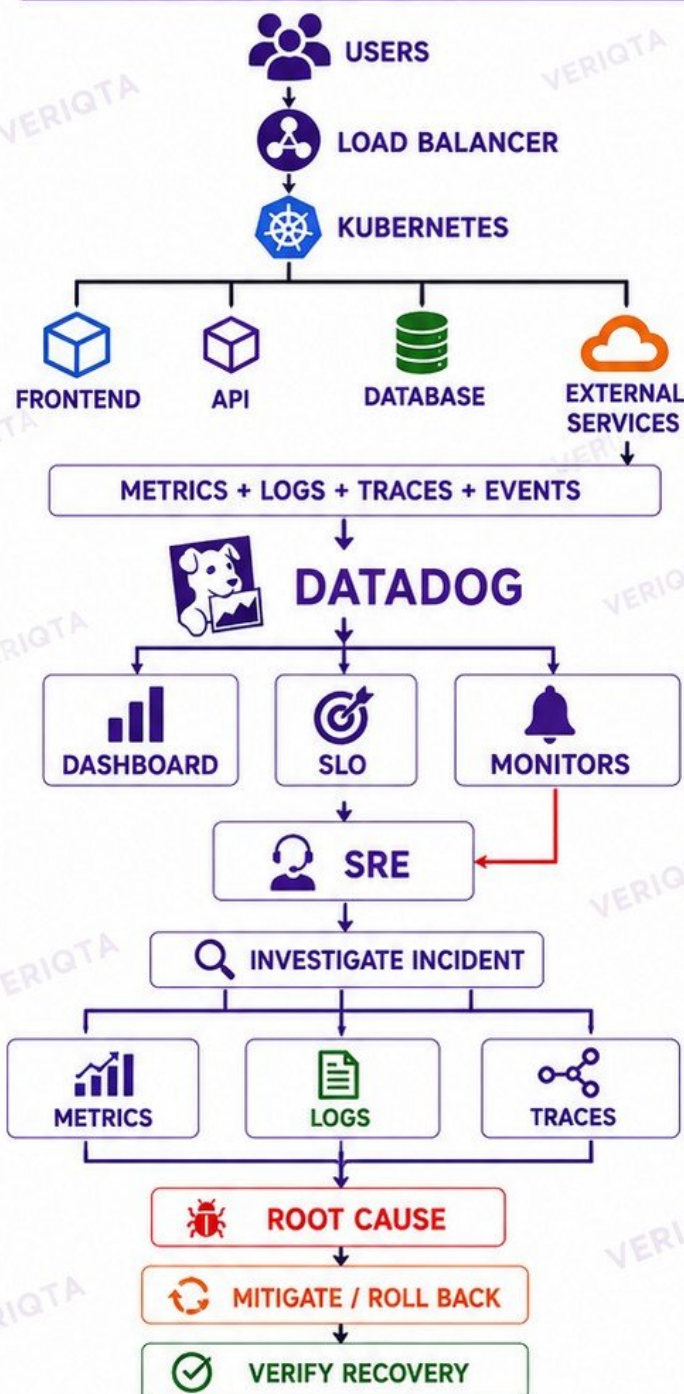
- 1  MONITOR A KUBERNETES APPLICATION
- 2  BUILD A GOLDEN SIGNALS DASHBOARD
- 3  DEFINE AN SLO
- 4  CONFIGURE ACTIONABLE MONITORS
- 5  DEPLOY A NEW APPLICATION VERSION
- 6  INTRODUCE LATENCY AND ERRORS
- 7  DETECT THE SLO DEGRADATION
- 8  IDENTIFY AFFECTED SERVICES
- 9  INSPECT INFRASTRUCTURE METRICS
- 10  ANALYZE TRACES
- 11  CORRELATE LOGS
- 12  CHECK KUBERNETES STATE
- 13  IDENTIFY THE FAULTY RELEASE
- 14  ROLL BACK
- 15  VERIFY RECOVERY
- 16  CONFIRM SLO / ERROR-BUDGET STATE
- 17  REVIEW THE INCIDENT



SRE TIP

The goal is not just to fix the incident, but to learn, improve, and prevent it from happening again.

FINAL ARCHITECTURE



SRE INCIDENT LIFECYCLE: DETECT → INVESTIGATE → MITIGATE → VERIFY → IMPROVE



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta